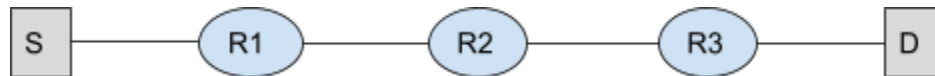


**CSE638: Graduate Systems**  
**Endsem Winter 2026 (SOLUTIONS)**  
**Date: April 27, 2026; Duration: 120 mins ( 9.30 am to 11.30 am)**  
**Total marks: 55**

**Instructions:**

- (1) Read the questions carefully and answer.
- (2) **Answer to the point as much as possible.** Include the important technical details.
- (3) If you make practical & appropriate assumptions, mention them clearly.

**Q.1. [CO1]**



Three routers are connected via a store-and-forward packet switch with **1 Mbps** links. Each link is **100 km** long and supports a **signal speed of  $2 \times 10^8$  meters per second**. The size of a packet is **10K bits**. **Ignore processing delays at the router and both the hosts.**

**[1+1+1+1+2 = 6]**

- (a) Compute frame transmission time
- (b) Compute propagation delay
- (c) What is the time elapsed between the transmission of the first bit of the packet at S and the reception of the last bit of the packet, at D?
- (d) We want to continuously send 1000 packets. What is the time elapsed between transmitting the first bit of the first packet at S and the reception of the last bit of the last packet at D?
- (e) The protocol requires the sender to receive an ACK after sending every 1000 packets. There are no losses/errors, and the size of **ACK packet is 1000 bits**. What is the maximum achievable throughput?

**Ans:**

$L = 10K$  bits,  $R = 1$  Mbps, Link\_length,  $l = 100$  km, Signal speed,  $s = 2 \times 10^8$  m/s,  $d_{proc} = 0$   
 $d_{trans}$  (10K bit packet) =  $L/R = 10$  ms  
 $d_{prop} = l/s = 0.5$  ms;  $[d_{prop} * 4 = 2$  ms allowed if you find the prop delay from S to D]  
 $t(S \text{ to } D) = (d_{trans} + d_{prop}) + (d_{proc} + d_{trans} + d_{prop}) * 3$   
 $= 10.5 + 10.5 * 3$   
 $= 42$  ms  
 $t(S \text{ to } D) = (1000 * d_{trans} + d_{prop}) + (d_{proc} + d_{trans} + d_{prop}) * 3$   
 $= (1000 * 10 + 0.5) + 10.5 * 3$   
 $= 10032$  ms  
 $d_{ACK} = 1000/10^6 = 1$  ms **[0.5]**  
 $t(D \text{ to } S) = (d_{ACK} + d_{prop}) + (d_{ACK} + d_{prop}) * 3 = 1.5 * 4 = 6$  ms **[0.5]**  
 Maximum achievable throughput = Total data sent/Total Time taken **[1]**  
 $= (1000 * 10K) / ((10032 + 6) * 10^{-3})$   
 $= 996.21$  Kbps

**Acceptable answers: 996Kbps, 996.2Kbps, 996.21Kbps**

**Q.2. [CO1]** The routing table of a router is shown below: **[1+1+1+1=4]**

| entry# | Destination | Subnet mask     | Interface |
|--------|-------------|-----------------|-----------|
| I      | 10.1.1.0    | 255.255.255.0   | 1         |
| II     | 10.1.1.128  | 255.255.255.128 | 2         |
| III    | 10.1.1.64   | 255.255.255.192 | 3         |
| IV     | 10.1.1.192  | 255.255.255.192 | 4         |
| V      | Default     |                 | 0         |

On which interface will the router forward packets addressed to the following destinations? Explain how you obtain the answer. **Note that you will NOT be awarded partial points for correct answers without explanation.**

- (a) 10.1.1.16
- (b) 10.1.1.72
- (c) 10.1.1.132
- (d) 10.1.10.191

**Ans:**

1

Matches with the entry I only; choose the interface for entry I

3

Matches with the entries I, and III; use longest prefix match (LPM); choose the interface for entry III

2

Matches with the entries I and II; use longest prefix match (LPM); choose the interface for entry II

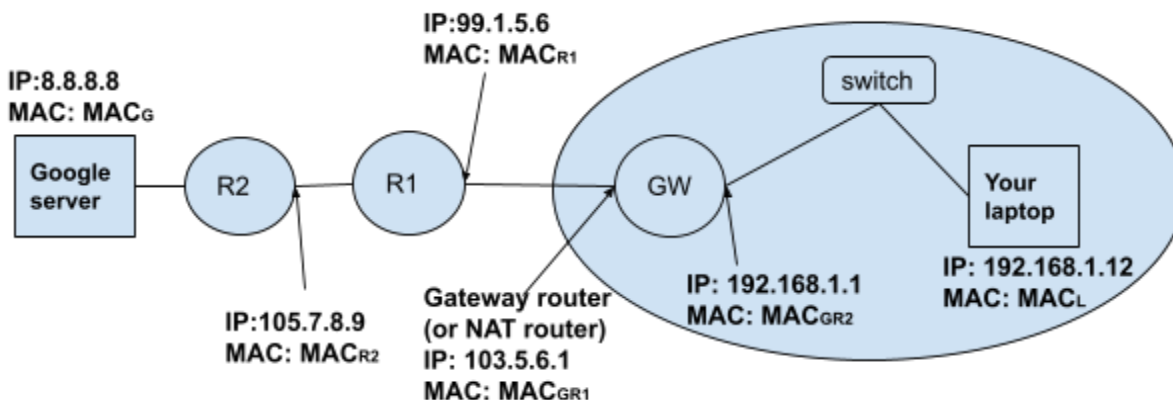
0

Does not match with any entry, choose the default interface

**Q.3. [CO1]** Suppose you are trying to access **google.in** from your laptop while you are in IIITD. As you might know IIITD provides NAT support. Suppose, you send a HTTP GET request and the server replies back with HTTP OK. Assume that there is no caching.

Write down the source IP, src port, destination IP and dest port number of the response packet. **[2]**

Now when the response reaches the Gateway router. The router forwards the packet to your laptop. Write down the source IP, src port, destination IP and port number of the response packet. **[2]**



Assuming when the first packet i.e., TCP SYN packet is sent from the client, the NAT creates a mapping between  $\langle 192.168.1.12, 5000 \rangle \Rightarrow \langle 103.5.6.1, 5001 \rangle$ . The private port of the client is 5000, and the public port is 5001

**Note:** The numbers 5000 and 5001 will be assumed by the students. The only condition is that these numbers should be greater than 1024

|   | Src IP  | Src port | Dst IP       | Dst port |
|---|---------|----------|--------------|----------|
| a | 8.8.8.8 | 80       | 103.5.6.1    | 5001     |
| b | 8.8.8.8 | 80       | 192.168.1.12 | 5000     |

**Q.4. [CO3]** Suppose a simple echo server application is running on a multicore machine, which responds with the current date/time to each request. Using some tools, we identified that the core on which the server is running needs to fetch each request from another core's cache. What could be the reason for such a case? How would you solve this problem? **[1+1 = 2]**

**Ans:**

- NIC delivers interrupts to the CPU core where the server application is not running
- Pin the server application and the interrupts to the same CPU core

#### Detailed explanation

##### **Producer–Consumer Core Mismatch**

If packets (or requests) are received on Core A (e.g., NIC interrupt, softirq, or kernel network stack), but processed by your server on Core B, then the request data (socket buffers, descriptors, etc.) are: first written into Core A's cache, then accessed by Core B, triggering cache line transfer (MESI invalidation + ownership migration).

Result: every request causes remote cache access / cache line bouncing.

##### **Interrupt (IRQ) Affinity Misconfiguration**

- RSS/RPS/XPS are not aligned with application threads
- IRQ affinity is left to default OS behavior

##### **Shared Queue / Lock-Based Design**

If requests are enqueued into a shared structure: lock-protected queue, shared ring buffer, then: enqueue happens on one core, and dequeue happens on another. This causes cache line bouncing on queue metadata, frequent ownership transfers

### NUMA Effects (if multi-socket)

If NIC is attached to NUMA node 0, and server thread runs on NUMA node 1, then memory backing the request lives “closer” to another socket → higher latency.

#### Solution:

- Core Affinity Alignment
- Flow Steering (RSS / RFS / XPS)
- Per-Core (Sharded) Design
- NUMA-Aware Placement
- Avoid Shared Data Structures

**Q.5. [CO2]** List TWO principles of system design (recall the first class where system design principles were discussed). How do these principles help? Under what conditions should you ignore implementing these principles? **[0.5\*2 + 0.5\*2 + 0.5\*2 = 3]**

Ans.

For example:

(i) Abstraction: Provide API/interfaces to use the code/module; Helps improves reusability and hides complexity; Adds API overheads → ignore when latency/speed is important.

(ii) Caching: Place frequently used data/objects close to the user, i.e., the application/processor that consumes this object; Reduces the latency for objects that are frequently read; However, if these objects are updated frequently and if these updates have to be reflected in real-time then caching increases the overhead.

**Q.6. [CO1]** Linux’s fork()+exec() model is elegant but increasingly considered expensive. For example, it requires duplicating page tables; copy-on-write still incurs overhead; it is a poor fit for multithreaded parents; it requires a high memory footprint with large address spaces (e.g., AI workloads).

(a) Explain the working of fork() and fork()+exec() system calls. **[1+1 = 2]**

(b) Design a replacement API for process creation optimized for large-memory workloads. What semantics of fork()+exec() would you preserve, and what semantics would you drop/modify? **[1+1 = 2]**

(c) Suppose you open the shell/terminal on your Linux machine and type the command,

**ls | grep txt**

How does the shell execute the above command? Specify the system calls used, in the exact order in which they are called. **[1.5]**

Ans.

(a)

**fork()** creates a new process by duplicating the calling (parent) process.

The child process gets a copy of the parent’s address space. Both processes continue executing from the next instruction. Returns twice: (i) parent gets child PID; (ii) child gets 0. Modern OSes use copy-on-write (COW), so that memory pages are shared until they are modified.

**exec()** replaces the current process’s memory with a new program. It does not create a new process; it overwrites the current process image, and the PID stays the same.

**Step-by-step processing: fork()+exec()**

- Parent calls `fork()`
- Child is created (duplicate)
- Child calls `exec()` to run a new program
- Parent optionally waits

(b)

**Design choices:**

Preserve:

- Ability to start a new program
- File descriptor inheritance (selectively)
- Parent-child relationship

Drop or modify:

- Full address space duplication
- Implicit inheritance of all memory
- Implicit signal inheritance

**Optimizations:**

- No address space copy
- Explicit resource inheritance bitmap
- Pre-initialized process pools

(c)

- `fork()` → can be ignored in the evaluation
- set up pipe → can be ignored in the evaluation
- `fork()` → again
- `exec(ls)`
- `exec(grep)`

**Q.7. [CO3]** What is the purpose of Amdahl's law? Suppose, as per Amdahl's law, an application is found to be highly sequential. What can you do to improve the performance of such applications? **[1+1 = 2]**

**Ans:**

Amdahl's law is used to estimate performance gains due to parallelism.

Possible answers:

- Identify why the code is sequential and fix it at the source. E.g., reduce co-ordination overhead, use lock-free data structures
- Improve Single-Threaded Performance: increase clock speed, micro-optimizations such as cache locality

BE CREATIVE!

**Q.8. [CO2, CO3]** A sender transmits data to a receiver over the TCP protocol. **[2+2 = 4]**

Given:

- Link bandwidth: 100 Mbps
- Round Trip Time, RTT: 50 ms
- Receiver advertised window (rwnd): 20 KB
- No packet loss unless specified

(a) What is the maximum achievable throughput for this system?

(b) Is the system flow-control limited or congestion-control limited?

**Ans:**

(a) Maximum achievable throughput is given by Bandwidth-Delay Product (BDP) [1]

$BDP = 100 \text{ Mbps} * 50 \text{ ms} = 5000 \text{ Kb} = 625 \text{ KB}$  [1]

(b) Receiver advertised window is,  $rwnd$  is 20 KB, which is much less than the network capacity, [1] i.e., 625 KB. Therefore, the system is limited by receiver buffer, i.e., flow-control limited. [1]

#### Q.9. [CO2, CO4]

A company is deploying a multi-tenant AI inference service with strict latency and scalability requirements. Its system components include:

- Client Applications: Send inference requests (text, images, multimodal inputs); expect low latency (50–200 ms) responses.
- API Gateway: Handles authentication, rate limiting, and routing.
- Inference Orchestrator: Decides: Which model to use? Which GPU cluster/node to route to? Maintains load balancing and scheduling.
- GPU Workers: Execute model inference. May run batching, caching, or model sharding.
- Monitoring & Logging System: Tracks latency, errors, and GPU utilization, and requires near-real-time updates.
- Autoscaling Controller: Dynamically scales GPU workers based on load.
- Analytics / Offline Training Pipeline: Consumes logs for model improvement and usage analytics

For each interaction below, **choose the most appropriate API paradigm (REST vs RPC/gRPC vs Publish–Subscribe)** and **justify your choice** [6]

(A) Client → API Gateway

(B) API Gateway → Inference Orchestrator

(C) Orchestrator → GPU Workers

(D) GPU Workers → Monitoring System

(E) Monitoring → Autoscaling Controller

(F) Logs → Analytics Pipeline

**Ans.**

| Interaction                              | API choice                                    | Justification                                                |
|------------------------------------------|-----------------------------------------------|--------------------------------------------------------------|
| (A) Client → API Gateway                 | <b>REST</b>                                   | Public-facing, interoperable, stateless                      |
| (B) API Gateway → Inference Orchestrator | <b>RPC/gRPC</b>                               | Low latency, efficient internal communication                |
| (C) Orchestrator → GPU Workers           | <b>Publish–Subscribe or RPC/gRPC (hybrid)</b> | RPC for request-response; Pub-Sub if async batching/queueing |

|                                         |                          |                                              |
|-----------------------------------------|--------------------------|----------------------------------------------|
| (D) GPU Workers → Monitoring System     | <b>Publish–Subscribe</b> | High-throughput streaming, decoupled logging |
| (E) Monitoring → Autoscaling Controller | <b>Publish–Subscribe</b> | Event-driven scaling decisions               |
| (F) Logs → Analytics Pipeline           | <b>Publish–Subscribe</b> | Scalable, multi-consumer data pipelines      |

#### Q.10. [CO2, CO4]

You are designing a low-latency trading platform with the following components:

- (1) Market Data Feed Handler: Receives live updates from exchanges (prices, trades, order book); Update rate: 1 million messages/sec
- (2) Strategy Engine: Consumes market data; Makes trading decisions in <10  $\mu$ s
- (3) Order Execution Gateway: Sends buy/sell orders to exchanges; Requires ultra-low latency (<5  $\mu$ s overhead)
- (4) Risk Management System: Validates trades (position limits, exposure)
- (5) Monitoring & Logging: Stores all events for compliance and debugging

**Choose Serialization Format:** Select between: JSON, Protobuf, and FlatBuffers for each of the following interaction. Justify your answer. **[4]**

- (A) Market Feed → Strategy Engine
- (B) Strategy Engine → Order Gateway
- (C) Order Gateway → Risk System
- (D) System → Logging/Audit

**Ans.**

| Interaction                         | API choice         | Justification                                                                                               |
|-------------------------------------|--------------------|-------------------------------------------------------------------------------------------------------------|
| (A) Market Feed → Strategy Engine   | <b>FlatBuffers</b> | Zero-copy → critical for microsecond latency; Avoids parsing overhead; Can directly access fields in memory |
| (B) Strategy Engine → Order Gateway | <b>FlatBuffers</b> | Ultra-low latency path; Serialization overhead must be near-zero                                            |
| (C) Order Gateway → Risk System     | <b>Protobuf</b>    | Slightly relaxed latency; Needs structured validation; Easier schema evolution than FlatBuffers             |
| (D) System → Logging/Audit          | <b>JSON</b>        | Human-readable logs; Regulatory/debugging needs; Latency less critical                                      |

**Q.11. [CO2, CO4]**

Suggest whether the following combinations are good or bad design choices. Justify your answer. Marks are only for correct justification. **[2\*3 = 6]**

1. REST + HTTP/1.1 + FlatBuffers
2. Pub-Sub + HTTP/1.1 + JSON for 10K clients
3. RPC + HTTP/3 + JSON for low-latency inference

**Ans.**

1. REST + HTTP/1.1 + FlatBuffers → BAD DESIGN CHOICE

Mismatch between REST and FlatBuffers

- REST APIs are designed for human-readable, loosely coupled interfaces.
- FlatBuffers is a binary, schema-driven, zero-copy format → not human-readable.
- Breaks REST's core strength: debuggability and interoperability.

Tooling & ecosystem limitations

- Browsers, curl, and most REST clients natively support JSON, not FlatBuffers.
- Requires custom clients → defeats REST's purpose.

HTTP/1.1 limitations

- No multiplexing → inefficient for modern APIs.
- Head-of-line (HOL) blocking → increases latency under load.

2. Pub-Sub + HTTP/1.1 + JSON for 10K clients → BAD DESIGN CHOICE

HTTP/1.1 does not scale for Pub-Sub

- Requires one TCP connection per client
- No multiplexing → inefficient for 10K subscribers
- High connection overhead and resource usage

JSON causes bandwidth explosion

- 2× payload size → amplified in fan-out
- With 10K clients:  
Bandwidth scales linearly with payload size × subscribers

Latency issues

- HTTP/1.1 lacks efficient streaming (relies on hacks like long polling)
- High latency and poor real-time performance

3. RPC + HTTP/3 + JSON for low-latency inference → BAD DESIGN CHOICE

JSON is the bottleneck

- High serialization/deserialization cost
- Adds milliseconds of latency, unacceptable for inference

Inefficient CPU utilization

- Parsing JSON at high QPS → CPU-bound system

Wasted benefits of HTTP/3

- HTTP/3 (QUIC) reduces transport latency and HOL blocking
- But JSON overhead dominates → transport gains are negated

**Q. 12. [CO4]**

A distributed key-value store uses consistent hashing to distribute keys across servers.

**[2.5+1+1+2+2 = 8.5]**

- Hash space: 0 to 999 (circular ring)
- Hash function:  $H(x) = x \bmod 1000$

Servers are placed on the ring using their hashed IDs.

Given:

- (ServerId, HashID) = {(S1, 100), (S2, 300), (S3, 600), (S4, 850)}
- Suppose incoming requests hash to the IDs as follows:
  - (RequestId, HashID) = {(R1, 50), (R2, 120), (R3, 320), (R4, 700), (R5, 900)}

(a) Assign each request (R1–R5) to the appropriate server on the ring.

(b) How many requests does each server process?

(c) Comment on whether the distribution is balanced

(d) A new server S5 with hash ID 400 is added. Recompute key assignments. Which keys are reassigned? How many requests moved?

(e) Suppose Server S3 (hash 600) fails. Recompute key assignments. Which keys are affected? How many requests moved?

**Ans.**

(a)

| Request | Hash | Assigned Server | Reason         |
|---------|------|-----------------|----------------|
| R1      | 50   | S1 (100)        | Next clockwise |
| R2      | 120  | S2 (300)        | Next           |
| R3      | 320  | S3 (600)        | Next           |
| R4      | 700  | S4 (850)        | Next           |
| R5      | 900  | S1 (100)        | Wrap around    |

(b, c) Load is almost balanced with server 1 serving two requests, and others serve 1 request.

(d) New server, S5 added → New assignments

| Request  | Old | New                |
|----------|-----|--------------------|
| R1 (50)  | S1  | S1                 |
| R2 (120) | S2  | S2                 |
| R3 (320) | S3  | S5 (moved request) |
| R4 (700) | S4  | S4                 |
| R5 (900) | S1  | S1                 |

(e) Server, S3, fails

| Request  | Old | New                |
|----------|-----|--------------------|
| R3 (320) | S3  | S4 (moved request) |

// NO REQUESTS WILL BE MOVED SINCE ALL S3 requests moved to S5 in (d)

Roll. No.: \_\_\_\_\_ Name: \_\_\_\_\_ Section: \_\_\_\_\_

\_\_\_\_\_ **THE END** \_\_\_\_\_